

# boot\_smoke

## boot\_smoke.sh — user guide

**Revision:** 1 **Last modified:** 2026-06-11T00:00:00Z

Companion documentation (§11.4.18) for tests/emulator/ab\_virt/  
boot\_smoke.sh.

### Overview

boot\_smoke.sh boots the base aarch64 Buildroot guest image produced by build\_image.sh under QEMU -machine virt + HVF on an Apple-Silicon host and **proves it reaches a live, interactive Linux userspace** — not a frozen frame (§11.4.107). A deterministic expect driver waits for the real buildroot login: prompt, logs in as root, runs uname -a and cat /etc/os-release, and prints a sentinel string that can only appear from an interactive shell after a successful login.

This is the foundation phase **PWU-AB-1** builds on (U-Boot bootcount + RAUC dm-verity slots).

**Proven status (FACT):** this smoke test has run GREEN. Evidence: docs/qa/20260611T061626Z-ab-virt-boot/console.log.

### Prerequisites

- A completed build\_image.sh run: out/.ok present and out/images/{Image, rootfs.ext2} non-empty (the script aborts with exit 3 otherwise).
- qemu-system-aarch64 on PATH with HVF support.
- expect on PATH.
- HELIX\_AB\_ROOT\_PW must match the password baked into the image by build\_image.sh (default helixota).

### Usage examples

```
# Run the boot smoke (after build_image.sh has produced a GREEN  
out/.ok).  
tests/emulator/ab_virt/boot_smoke.sh
```

```
# With a non-default root password (must match the built image).
HELIX_AB_ROOT_PW=helixota tests/emulator/ab_virt/boot_smoke.sh
```

## Inputs

| Input                                                     | Source                              | Default  |
|-----------------------------------------------------------|-------------------------------------|----------|
| HELIX_AB_ROOT_PW                                          | env → ROOT_PW                       | helixota |
| out/.ok, out/images/<br>Image, out/images/<br>rootfs.ext2 | filesystem (from<br>build_image.sh) | required |

The run id is derived from a UTC timestamp: <YYYYmmddTHHMMSSZ>-ab-virt-boot.

## Outputs

| Output                                  | Meaning                                                          |
|-----------------------------------------|------------------------------------------------------------------|
| docs/qa/<run-id>/console.log            | full QEMU boot transcript (the captured evidence, §11.4.83)      |
| docs/qa/<run-id>/<br>console.log.driver | the expect driver's own stdout/stderr                            |
| stdout PASS/FAIL lines                  | one [PASS]/[FAIL] per anti-bluff assertion + a final RESULT line |
| exit code                               | 0 = PASS, 1 = FAIL, 3 = precondition abort                       |

## Side-effects

- Spawns a transient qemu-system-aarch64 guest; it exits on guest poweroff or the expect timeout (self-cleaning, §11.4.14).
- Creates the docs/qa/<run-id>/ evidence directory.
- Does **not** modify the image artifacts (rootfs is mounted rw but the guest is powered off cleanly).

## Internal behaviour

1. set -u + set -o pipefail; resolve SCRIPT\_DIR, repo root, root password, image dir, run id, and evidence/console paths; mkdir -p the evidence dir.
2. **Preconditions** (each → exit 3 on failure): out/.ok present; Image + rootfs.ext2 non-empty; qemu-system-aarch64 found; expect found.
3. Run a deterministic expect driver (timeout 90s) that:
  - spawns QEMU -M virt -accel hvf -cpu host -smp 2 -m 512 -nographic with the kernel, root=/dev/vda console=ttyAMA0, and the rootfs as a virtio-blk-device;

- waits for `buildroot login:`, sends `root`, waits for `Password:`, sends the root password, waits for the `#` shell prompt;
  - runs `uname -a, cat /etc/os-release | head -2`, then echo `HELIX_USERSPACE_LIVE_OK`, then `poweroff`;
  - on any missing prompt the driver prints `HELIX_DRIVER_FAIL: ...` and exits 2. The transcript is captured via `log_file` into `console.log`.
4. **Anti-bluff assertions** — `grep` the captured `console.log` for all of:
- Booting Linux on physical CPU `.*0x610f` (kernel booted on the real Apple CPU via HVF);
  - Welcome to Buildroot (userspace init reached);
  - `buildroot login:` (getty login prompt presented);
  - `Linux buildroot .* aarch64 GNU/Linux` (post-login `uname -a` returned a live aarch64 kernel string);
  - `HELIX_USERSPACE_LIVE_OK` (the interactive sentinel printed **after** login — proves a live shell, not a frozen frame).
5. If every assertion holds, print `RESULT: PASS` + the evidence path and exit 0; otherwise print `RESULT: FAIL` (with the `qemu/expect rc`) and exit 1.

## Edge cases

- **out/.ok absent / images missing** → exit 3 (“run `build_image.sh` first”).
- **QEMU or expect not installed** → exit 3.
- **No login / password / shell prompt within 90s** → the driver emits a `HELIX_DRIVER_FAIL` line and exits 2; the corresponding `grep` assertions then `FAIL` and the script exits 1.
- **Poweroff times out** → the driver treats it as a non-fatal note (`HELIX_DRIVER_NOTE`) and exits 0; the verdict still depends solely on the captured-evidence assertions.

## Related scripts

- `tests/emulator/ab_virt/build_image.sh` — produces the Image + `rootfs.ext2` this script boots. See `docs/scripts/build_image.md`.
- `tests/emulator/ab_virt/assemble_ab_disk.sh` — the next phase: a 2-slot GPT disk for the real slot-switch boot. See `docs/scripts/assemble_ab_disk.md`.
- `docs/research/rk3588_emulator/REPORT.md` — the emulator design report.

## Last verified date

2026-06-11 — documented against the script as committed; PASS evidence at `docs/qa/20260611T061626Z-ab-virt-boot/console.log`.